

Retrieval Augmented Generation (RAG)

(inexpensive/
very easy/

outcomes may be poor)

Few-shot Prompting
(in-context learning
different from gradient
learning)

{ Example-1

Example-2

⋮

Example-n

}

..... prompt

LLM

outcomes are good

↓ (takes more time
for training)

Fine-tuning

(Large data
from 10,000 - 1M)

Full finetuning

Parameter
Efficient

Fine
Tuning (PEFT)

RAG

Extends the
knowledge base of
LLM

Why RAG?

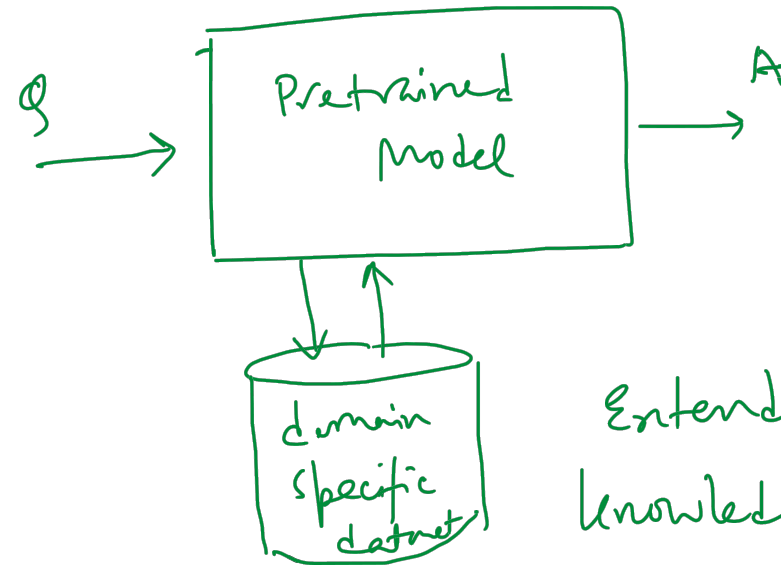


- will not have access to the recent data.
- will not have access to non-public documents (company internal documents)
- will not have access to domain specific data

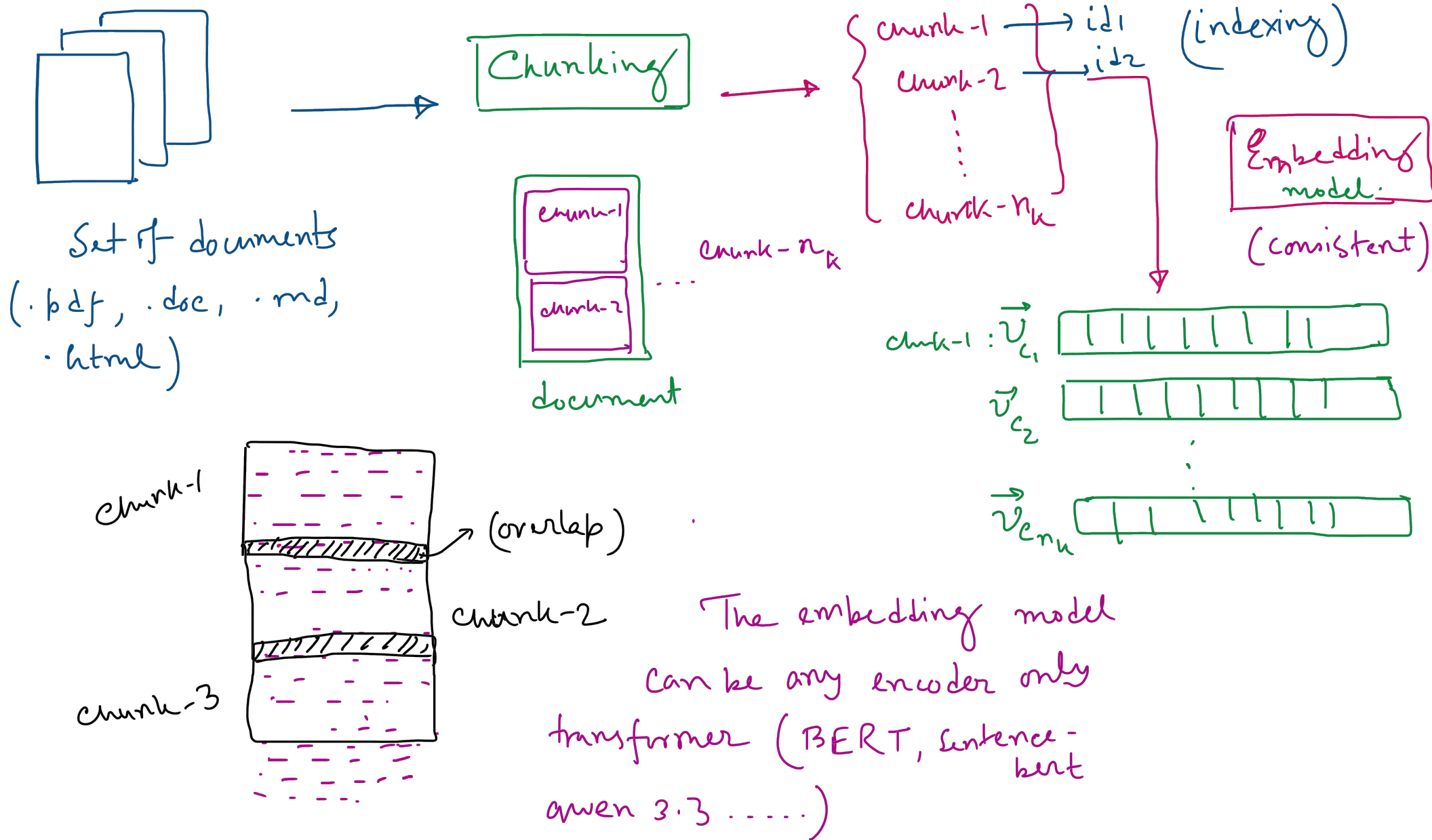
Pre-trained models are usually trained with a lot of training data from internet & other sources.

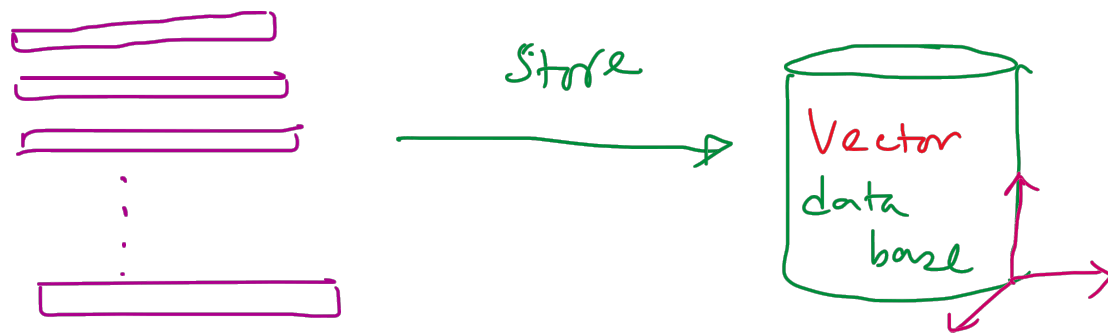
↓
publicly available data

(time window till which the data was available to train the model)

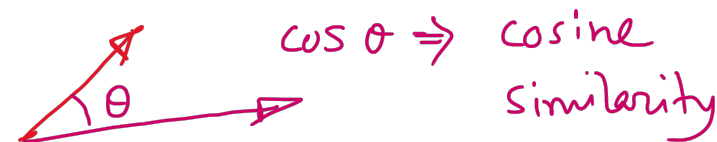


Extending the knowledge base of the pretrained model.





What is a vector database?



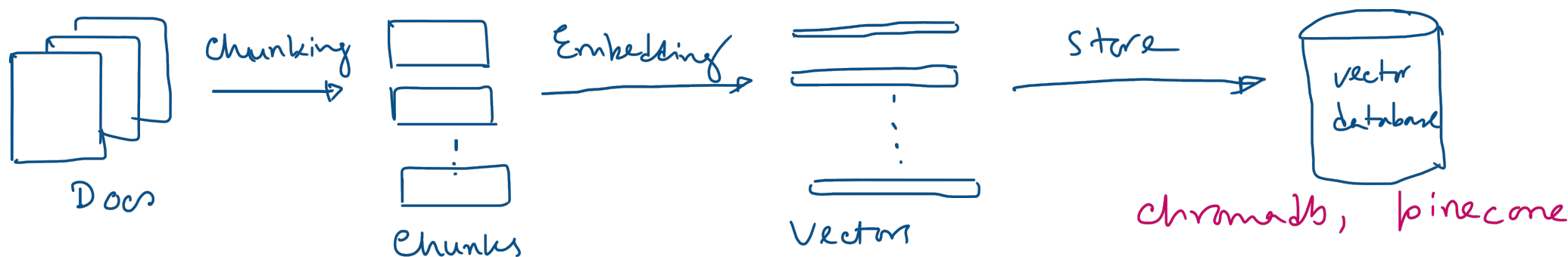
Embeddings
of chunks

`{'id': [.....]}`

FAISS : open source
vector
database

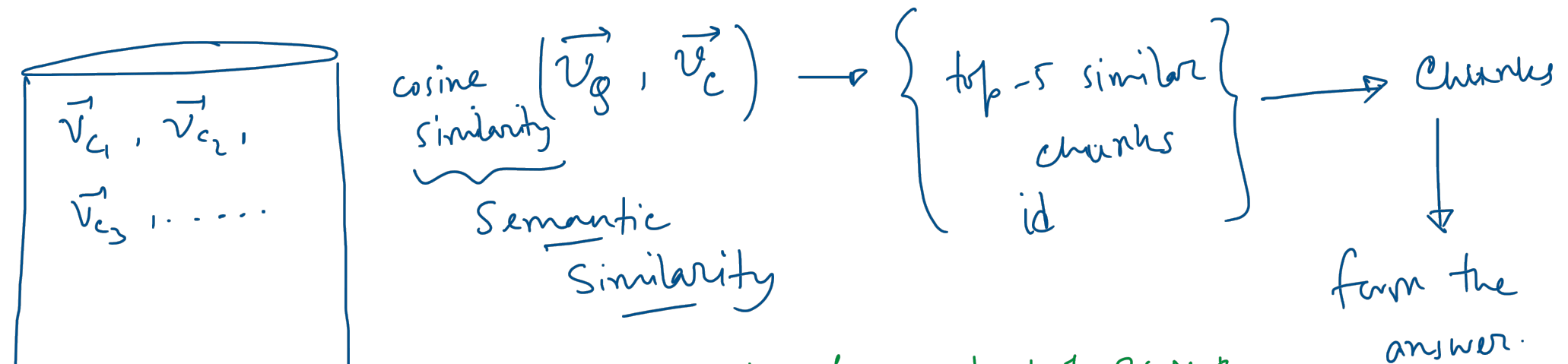
RDBMS stores the data in tabular format in tables

Where as, vector DB stores the vector (no-sql database)



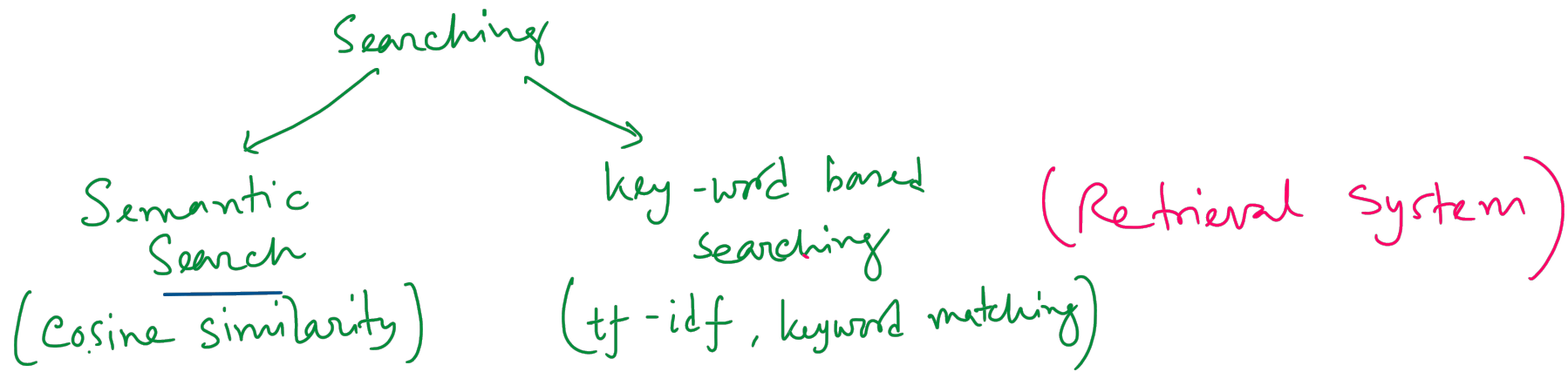


(same as the one
you used to create embeddings of
chunks)



the corporation has a debt of 36 M \$.

there is a xyz company which earns 20 M \$ from debt investment.



Hybrid Search

Reciprocal Rank Fusion (RRF)

(Rank from Semantic Search)

1. chunk-3
2. chunk-7
3. chunk-12
- ⋮

(Rank from keyword search)

1. chunk-12
2. chunk-9
3. chunk-3
- ⋮

$$\underline{RRF(d)} = \sum_{i=1}^K \frac{1}{\text{rank}_i(d) + c}$$

Reciprocal Rank Fusion (RRF)

	<u>Retrieval System-1</u>	<u>Retrieval System-2</u>	...	<u>Retrieval System-k</u>
(d).	<u>3</u>	8	...	2

in 3 retrieval systems ($K=3$) the rank of the document 'd' is 3, 8 and 2 respectively

$$RRF(d) = \sum_{i=1}^{K=3} \frac{1}{rank_i(d) + C}$$

$rank_i(d)$ → rank of the document in i th retrieval system.

C → the constant (60) usually

$$RRF(d) = \frac{1}{3+60} + \frac{1}{8+60} + \frac{1}{2+60} = \frac{1}{63} + \frac{1}{68} + \frac{1}{62} = 0.046$$

Nearest-Neighbour Search:

$\vec{v}_q$
query embedding

$$\vec{v}_{c_1}, \vec{v}_{c_2}, \dots, \vec{v}_{c_k}$$

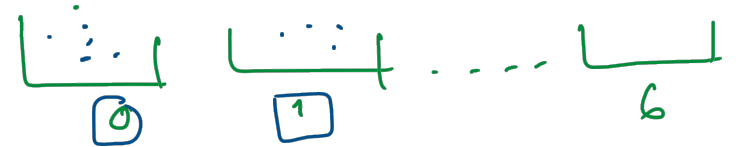
$$\|\vec{v}_i\| = d \quad O(\text{cosine similarity})$$
$$= O(d)$$

Approximate nearest neighbour search

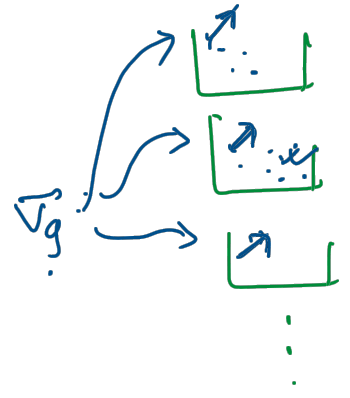
$\therefore O(\underline{c_k} \cdot \underline{d}) \rightarrow$ overall complexity of finding the semantic similarity of query vector with chunk embeddings.

Hashing
(%7)

$$\begin{array}{lll} 12 \xrightarrow{\%7} 5 & \longrightarrow & \text{bucket-5} \\ 14 \xrightarrow{\%7} 0 & \longrightarrow & \text{bucket-0} \\ 27 \xrightarrow{\%7} 6 & \longrightarrow & \text{bucket-6} \end{array}$$



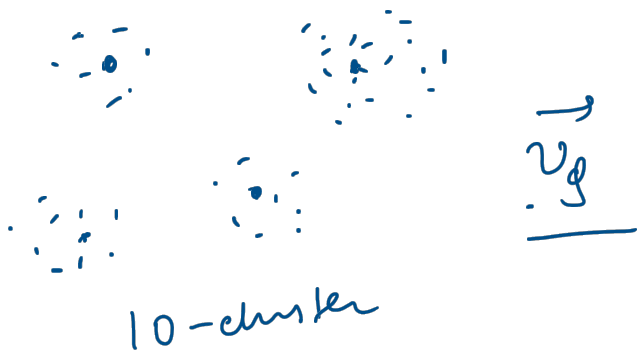
Hashing



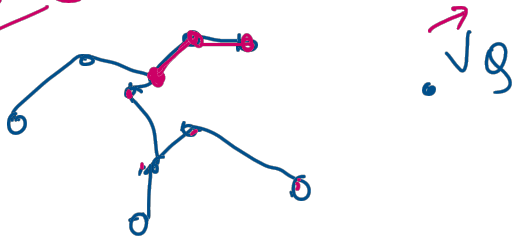
vectors which are closer to one another will result in same bucket.

(Locality Sensitive Hashing)

Cluster



HNSW (Graph based)



HNSW (Hierarchical Navigable Small World)

